

Project Proposal Template for Software Engineering Seminar

Project Title: Dormitory Parcel Management System

Team Members: 6631503034 Phuriphat Chantiuaong
6631503040 Sadayu Suriya
6631503042 Supawan Kongsapcharoen
6631503103 Kongphop Ruentongdi
6631503106 Danaiphong Songsikhwa

Date: 25/08/2026

1. Executive Summary

This project aims to develop a dormitory parcel management system capable of handling the dormitory's daily package volume, which currently averages approximately 418 incoming and 418 outgoing parcels per day (based on data recorded 15–20 August 2026), surging to as many as 1,024 items per day during Flash Sale periods. The current workflow — built on Google Forms and Google Sheets — suffers from significant bottlenecks, including slow data loading during peak hours, failed searches when parcels arrive without a visible room number, and mistaken hand-offs when room numbers are misread or swapped. The proposed system optimizes data entry, storage, and retrieval performance while strictly preserving a user interface similar to the existing spreadsheet-based workflow, so that operational staff can continue working without a steep learning curve. The system also introduces flexibility to the check-out process, allowing staff to check out all of a resident's pending parcels at once while retaining the option to check them out one at a time.

2. Introduction and Problem Statement

Dormitory parcel operations become highly congested during peak periods, such as Promotion/Flash Sale days or during the late afternoon and evening hours. In the current check-in process, staff must locate the resident's room number, write it on the box by hand, open a Google Form, enter the room number, scan the parcel's barcode, and submit. Parcels are then sorted and placed on shelves across 10 buildings, with oversized items stored together in a common area. For check-out, staff search for the resident's room number in a Google Sheet, count how many parcels are recorded, physically retrieve them, open a Google Form to scan and submit, and finally hand the parcels over.

The key issues can be summarized as follows:

1. **Identification Failures:** Many parcels arrive without a room number written on them. Searching by the recipient's name often fails because students use nicknames on their delivery labels, which do not match the official student records.
2. **Human Error in Labeling:** Staff handwriting on the boxes is often unreadable, or the recorded room numbers are unclear, making later retrieval difficult.
3. **System Inefficiency:** The current spreadsheet-based system suffers from slow data loading, which significantly delays the workflow during busy hours.
4. **Misplaced Storage and Retrieval Failures:** Parcels are sometimes shelved in the wrong location during sorting. When a resident later asks for it, staff cannot find it

even though it was recorded as received, forcing repeated searches or turning residents away empty-handed.

5. **Delivery Errors:** Room numbers occasionally get swapped during handling, resulting in parcels being handed to the wrong resident. This creates a complicated situation, as staff must wait for the incorrect recipient to return the item before it can reach its rightful owner — a special case that is time-consuming to resolve.

To address these challenges, we propose redesigning the dormitory's parcel management workflow into a dedicated web-based system. The system centralizes parcel records in a structured database, validates room numbers against the official resident directory at the point of entry to reduce name-mismatch errors, and is optimized for fast search and retrieval even under peak-volume conditions such as Flash Sale days. Critically, the new system preserves the visual layout and step-by-step flow that staff already use today, so it can be adopted with minimal retraining and disruption to daily operations.

3. Project Goals and Objectives

- **Primary Goal:** To develop a highly efficient and responsive dormitory parcel management system that resolves the current data-processing bottlenecks while preserving a familiar interface and workflow for operational staff.
- **Specific Objectives:**
 1. To optimize system performance and data retrieval speed, directly addressing the existing problem of slow data loading during peak parcel operations (including Flash Sale surges of up to 1,024 parcels/day).
 2. To reduce identification failures by validating room numbers and resident names against an official directory at check-in, rather than relying on free-text entry and handwriting.
 3. To design and maintain a user interface (UI) and user experience (UX) that closely resembles the existing spreadsheet-based workflow, ensuring staff can transition smoothly without a steep learning curve.
 4. To develop a flexible check-out module that allows staff to check out all of a resident's pending parcels simultaneously, while strictly retaining the option to process items individually in cases where a specific parcel cannot be found or is not fully collected by the resident.

4. Scope of Work

- **4.1. In-Scope Functionalities:**
 - **Module/User Role — Dormitory Staff (Front Desk/Operator):**
 - **Parcel Check-In:** Record incoming parcels by scanning the tracking barcode and selecting/validating the room number against the resident directory (autocomplete/dropdown instead of free-text), then assign a storage location (building/shelf).

- **Parcel Storage Tracking:** Tag each parcel with its physical storage location (by building, out of the 10 dormitory buildings, and shelf/bin) to prevent misplaced-item searches.
- **Parcel Check-Out:** Search by room number to view all pending parcels for that resident; check out all items in a single action, or select individual items to check out one at a time.
- **Search & Lookup:** Quickly search parcels by room number, tracking code, or resident name, with results returned even under high concurrent load.
- **Core System Features:** A dashboard for daily parcel volume and pending-item counts, plus fast, validated check-in/check-out workflows.
- **4.2. Out-of-Scope Functionalities:**
 - Automated Handwriting Recognition (OCR) of hand-written labels.
 - Mobile application (iOS/Android) — the system will be delivered as a web application only.
 - Direct API integration with courier companies for automatic tracking-status updates.

5. Methodology and Technical Approach

5.1. Development Methodology: Agile Scrum will be used to allow iterative development with stakeholder (dormitory staff/advisor) feedback every two weeks. This ensures flexibility to adapt based on real operational needs while maintaining a structured delivery schedule.

5.2. System Architecture: Three-Tier

- **Frontend (Client):** User-facing interface for check-in, check-out, and search.
- **Backend (Application Server):** API and business logic, including room-number validation and barcode handling.
- **Database Layer:** PostgreSQL, chosen for its strong support of the relational structure of the data (residents, rooms, parcels, staff, and status), which enables reliable constraints and fast indexed search as data volume grows.

5.3. Technology Stack:

- **Frontend:** React.js (UI), Tailwind CSS (styling)
- **Backend:** Go (Gin framework)
- **Database:** PostgreSQL
- **Deployment:** Docker

6. Risk Assessment and Mitigation Plan

Risk	Likelihood (Low, Med, High)	Impact (Low, Med, High)	Mitigation Strategy (Specific Actions)
Technical Risk - Learning curve with Go (Gin), React, PostgreSQL, and Docker integration	Med	Med	Allocate early “spike” weeks to prototype backend–frontend communication. Create small proof-of-concept apps before full implementation. If integration issues persist, fall back to simpler deployment (direct hosting without Docker).
Performance Risk - Slow loading or timeouts during high-volume periods (up to 1,024 parcels/day during Flash Sale)	Med	High	Add database indexing on room number and tracking code, implement pagination for list views, and load-test the check-in/check-out endpoints against simulated peak volume before go-live.
Data Risk - No production data available for testing (per dormitory's stated constraint)	Med	High	Generate synthetic parcel and resident datasets that mirror the real schema and observed volume patterns (~400–1,000 records/day). Validate the synthetic data's realism with dormitory staff before final testing, and migrate real historical Google Sheets data only at cutover.
User Adoption Risk - Staff resistance to changing from the familiar Google Sheets/Forms workflow	Med	Med	Design the UI to closely mirror the existing spreadsheet layout, run hands-on training sessions, gather staff feedback during UAT, and keep the old system available in parallel for a short transition period.
Scope Creep - Adding extra features beyond agreed scope	Med	High	Maintain a strict scope list (check-in, check-out, search, dashboard). Any new requests go through advisor review before implementation. Track scope in Git issues and weekly updates.
Team Risk - Member unavailability or workload imbalance	Low	High	Use GitHub for version control with clear commit history. Maintain updated technical documentation. Conduct weekly check-ins and share knowledge across members so no task depends on one person only.
Timeline Risk - Delay in development or testing	Med	Med	Build buffer weeks (polish & contingency phase). Prioritize critical modules (check-in, check-out, search) first, then add

Risk	Likelihood (Low, Med, High)	Impact (Low, Med, High)	Mitigation Strategy (Specific Actions)
			enhancements. Keep advisor updated on progress during sprint reviews.
Deployment Risk - Issues running Docker in the final environment	Low	Med	Test Docker deployment early in the development phase. Prepare a backup plan with cloud hosting (Heroku, Vercel, or Render). Document the deployment process clearly to avoid last-minute issues.

7. Milestones and Timeline

Milestone	Key Tasks	Reporting / Deliverable	Est. Hours	Timeline
1 - Analysis & Design	Kick-off meeting and initial topic brainstorming (19/08/2026); AI Ethics, PDPA & IT Law workshop and rule.md case (Week 2, 19/08/2026); conduct interviews with dormitory parcel staff and residents (21/08/2026); divide requirement-analysis and documentation tasks among team members (22/08/2026); research current dormitory parcel workflows and pain points; draft Software Requirement Specification (SRS) outline.	Requirement checklist, interview notes, and internal team review meeting (26/08/2026); Product Backlog + spec submitted to GitHub repo (Week 3 course deliverable).	40	End of Week 3 (26/08/2026)
2 - UX/UI Design	Create wireframes, design mockups (Staff/Operator roles) that closely mirror the existing spreadsheet layout, finalize check-in/check-out user flow diagrams; feature-list and user-journey design draft (Week 4).	Design draft submitted to GitHub repo (Week 4); UX/UI prototypes reviewed with advisor; DISCOVER Phase Gate submission (proposal + backlog + design + compliance review + ≥15 real users).	60	End of Week 5 (09/09/2026) – DISCOVER Phase Gate

Milestone	Key Tasks	Reporting / Deliverable	Est. Hours	Timeline
3 - Core Workflow Prototype	Scope lock, Sprint 0 / repo setup (Week 6), and architecture design studio (Week 7); implement barcode-scan check-in flow with room-number validation against the resident directory. Build a prototype search function for room number / tracking code / resident name. Benchmark search and retrieval performance under simulated load.	In-class architecture case review; prototype demo and short report on scan reliability and search performance, ahead of the Midterm Exam.	80	End of Week 7 (23/09/2026), before Midterm Exam (28/09–04/10/2026)
4 - Core Development	Backend APIs with Go (Gin), integrate PostgreSQL, React frontend (check-in dashboard, check-out dashboard, search); UI/UX build sprint.	Sprint reviews and Git repo updates; Alpha Demo Day presentation – BUILD Phase Gate.	200	End of Week 8 (07/10/2026) – BUILD Phase Gate (Alpha Demo Day)
5 - Testing & QA	Unit & integration testing (Testing Pyramid, Week 9), load testing against peak volume, user acceptance testing, bug fixing; incorporate AI Governance and IT/AI Law guest-lecture guidance (Week 10).	UAT feedback report; Beta Review Day demo and impact metrics – TEST Phase Gate.	60	End of Week 11 (28/10/2026) – TEST Phase Gate (Beta Review Day)
6 - Documentation & Finalization	Finalize SRS + User Manual, polish UI/UX, integrate feedback from the Beta Review; prepare materials for real-user validation.	Documentation and advisor check-in; real-user validation session with dormitory staff/residents (offsite).	40	Week 12 (04/11/2026) – Real-user validation
7 - Deployment & Delivery	Deploy system with Docker, prepare final presentation, finalize Git repo handover.	Final deployment and Final Project Presentation (Weeks 13–14); project and portfolio settle ahead of the Final Exam.	30	Weeks 13–14 (11–18/11/2026), through Final Exam period (22/11–04/12/2026)

8. Ethical Considerations

8.1. Data Privacy and Security: The system will enforce strict security protocols to protect resident data (names, room numbers, contact details). All communication will use secure HTTPS connections, staff passwords will be hashed and salted, and PostgreSQL will be configured with encryption at rest and in transit. Access will be controlled by a role-based system, ensuring staff only see information relevant to their responsibilities.

8.2. Accountability and Chain of Custody: Because misdelivered parcels are a known operational risk, every check-in and check-out action will be logged with a staff ID and timestamp, creating an audit trail that can be used to resolve disputes when a parcel is handed to the wrong resident or reported missing.

8.3. Data Retention: Parcel records will be retained only as long as operationally necessary (e.g., a defined number of months after pickup) and then archived or removed, in line with data-minimization practice for resident information.

8.4. Accessibility and Usability: The frontend will be designed for dormitory staff of varying technical proficiency, with a simple, low-typing workflow (autocomplete/dropdown room selection and barcode scanning rather than free-text entry), sufficient color contrast, and keyboard navigation support, so the system remains easy to use during high-pressure peak periods.

9. Expected Deliverables

1. A fully functional and deployed web application: the Dormitory Parcel Management System
2. The complete source code managed in a Git repository.
3. Comprehensive technical documentation, including a Software Requirements Specification (SRS) and a User Manual.
4. A final presentation and live demonstration of the system.

10. Conclusion

This project addresses a pressing operational challenge: the slow, error-prone process of managing hundreds of dormitory parcels a day using Google Forms and Sheets. By centralizing parcel records, validating room numbers at the point of entry, and preserving the interface staff already know, the system aims to reduce identification failures, misplaced items, and mistaken hand-offs while remaining fast even during Flash Sale surges. With a practical technical foundation, clear ethical safeguards, and a structured timeline, the project is both feasible and impactful, offering real operational value to the dormitory and significant learning opportunities for the student development team.

11. Immediate Next Steps (Optional but Recommended)

Upon approval of this proposal, our team will immediately undertake the following actions to ensure a swift and well-structured kickoff for the project:

- **Establish Project Management Tools:** Set up a project board (e.g., Jira, Trello, or GitHub Projects) populated with the initial epics and user stories defined in Milestone 1.
- **Initialize Version Control:** Create a private Git repository, configure branching policies, and prepare the initial backend/frontend structure for Go (Gin) and React.
- **Dataset Preparation:** Since the dormitory cannot provide production data for testing, generate synthetic parcel and resident datasets that reflect the real daily and peak-day volume patterns (~417 parcels/day, up to 1,024 during Flash Sale) to support early development and testing.
- **Schedule Kickoff Meeting:** Conduct a formal team kickoff meeting to finalize the backlog for Sprint 1, assign responsibilities, and align development milestones.

12. References

Docker Inc. (n.d.). Docker Documentation. Retrieved September 2, 2026, from <https://docs.docker.com/>

Meta Platforms, Inc. (n.d.). React: The library for web and native user interfaces. Retrieved September 2, 2026, from <https://react.dev/>

PostgreSQL Global Development Group. (n.d.). PostgreSQL: The World's Most Advanced Open Source Relational Database. Retrieved September 2, 2026, from <https://www.postgresql.org/>

Tailwind Labs Inc. (n.d.). Tailwind CSS Documentation. Retrieved September 2, 2026, from <https://tailwindcss.com/>

Gin Web Framework. (n.d.). *Gin-gonic: HTTP web framework written in Go (Golang)*. Retrieved September 2, 2026, from <https://gin-gonic.com/>